

A system that uses privacy will use encryption algorithms like Extended Tiny Encryption Algorithm (XTEA) or Advanced Encryption Standard (AES). These are industry-standard encryption algorithms that are being used in a wide variety of electronic devices and are being considered secure enough for a large number of applications. These algorithms can be implemented either in hardware or in software. There are different opinions (even among specialists) regarding which implementation, whether hardware or software, is the more secure one.

Adding higher encryption adds to cost

In general, a stronger security level will result in an increase in the overall cost of the system. Both hardware and software implementation will have an impact. Even a complete software implementation will add additional cost; this is because the firmware also needs to occupy more memory. The more memory is required, the bigger is the additional cost.

As mentioned earlier, some implementations use a hardware encryption module, while others use software implementations. But essentially, both implementations add additional cost. Software implementations have bigger requirements on both program and data memory. On the other hand, hardware implementation has bigger requirements in terms of the size of the silicon die. There are pros and cons to both sides. The choice is up to the system designer.

But even if the designer is using a very high level of security, this does not lock out attacks like capture-and-replay or jamming and selectively replaying information.

Typical security threats

This section introduces you to some of the modern threats to secur-

ing your system and methods to secure these.

Brute force attack. The brute force attack is a very basic cryptanalytic attack that involves using all possible combinations until the correct one is reached. Of course, this is more a theoretical approach rather than a real implementation, and depending on the number of possible combinations, it can take a long time to complete. With modern encryption techniques, such an attack would take many years to complete.

Capture and replay. This attack is one in which valid data is fraudulently captured and re-transmitted. There are several countermeasures to avoid such attacks. One such measure is using session tokens. These tokens are used as a part of the encryption system. Each token is valid only for a particular session; the token (and the transmissions derived from that token) cannot be used in another session. One-time passwords are very similar to session tokens except that these are used to authenticate individual transactions in a session. However, these are difficult to be implemented in one-way systems.

Check the integrity of your data. The message authentication code is a short piece of information added to a data transaction and used to authenticate a message to provide data integrity. Data integrity assures the detection of accidental and intentional message changes, while authenticity assures the message's origin. This does not provide the privacy aspect but more the authentication.

Note the time. Time-stamping is another way of preventing a capture-and-replay attack. It involves sending periodic time-stamped messages. At the receiver end, the time-stamp is compared to a local estimated one within a reasonable tolerance

level. This poses some special design challenges.

A time-keeping source

To overcome some of today's major security threats, we propose a system that features the concept of limited time valid data packet. In other words, every data packet that the transmitter/encoder is sending over the air has time-limited validity. Even though, in essence, the data packet itself is perfectly valid in terms of proper encryption and data contained, it is valid only for a very limited time period. A capture-and-replay attack would have very little success in attacking such a system. This is because such an attack involves capturing a data packet and then re-transmitting it at a later time; but the data packet is only valid for the time frame for which it was transmitted.

To implement this, each data packet needs to include a time-stamp. Thus, the transmitter needs to have a time-keeping source. The easiest way to achieve this is by using a crystal oscillator. (A 32.768kHz crystal is a good choice.) Today's microcontrollers feature crystal-driven time-keeping peripherals that can work even in low-power states. Thus, the processor can keep track of time even in sleep mode. Any other time-keeping method can be used as well, provided that it features good stability over time and temperature.

At the other end. On the decoder side, a similar time-keeping mechanism is used (if not the same). The first time an encoder is paired with a decoder, the latter stores information such as the serial number and its timer value. It then stores a delta time value that represents the time difference between the encoder and the decoder onboard timers. At this time, the encoder is synchronised with the decoder. At a later time, when a transmission from the encoder is